



MCSE699J- Dissertation II

AGENTOPS AI: SMART SERVICE AVAILABILITY AND LOG ANOMALY DETECTION SYSTEM.

Submitted in partial fulfilment of the requirements for the degree of

Master of Technology

in

Computer Science and Engineering (Artificial Intelligence and Machine Learning)

by

24MAI0027 DEEPTI SHARMA

Under the Supervision of

Prof. Manjula R

Professor Grade 1

School of Computer Science and Engineering



VIT[®]
Vellore Institute of Technology
(Deemed to be University under section 3 of UGC Act, 1956)

April, 2026

DECLARATION

I hereby declare that the dissertation entitled “AGENTOPS AI: SMART SERVICE AVAILABILITY AND LOG ANOMALY DETECTION SYSTEM.” submitted by me, for the award of the degree of Master of Technology in Computer Science and Engineering (Artificial Intelligence and Machine Learning) to VIT is a record of bonafide work carried out by me under the supervision of Prof. Manjula R, School of Computer Science and Engineering.

I further declare that the work reported in this dissertation has not been submitted and will not be submitted, either in part or in full, for the award of any other degree or diploma in this institute or any other institute or university.

Place: Vellore

DEEPTI SHARMA (24MAI0027)

Date :

CERTIFICATE

This is to certify that the dissertation entitled “AGENTOPS AI: SMART SERVICE AVAILABILITY AND LOG ANOMALY DETECTION SYSTEM.” submitted by DEEPTI SHARMA (24MAI0027), School of Computer Science and Engineering, VIT, for the award of the degree of Master of Technology in Computer Science and Engineering (Artificial Intelligence and Machine Learning), is a record of bonafide work carried out by the student under my supervision during Winter Semester 2025-2026, as per the VIT code of academic and research ethics.

The contents of this report have not been submitted and will not be submitted either in part or in full, for the award of any other degree or diploma in this institute or any other institute or university. The dissertation fulfills the requirements and regulations of the University and in my opinion meets the necessary standards for submission.

Place: Vellore

Signature of the Guide

Date :

Internal Examiner

External Examiner

Head of the Department

ACKNOWLEDGEMENT

I DEEPTI SHARMA (24MAI0027) am deeply grateful to the management of Vellore Institute of Technology (VIT) for providing me with the opportunity and resources to undertake this project. Their commitment to fostering a conducive learning environment has been instrumental in my academic journey. The support and infrastructure provided by VIT have enabled me to explore and develop my ideas to their fullest potential.

My sincere thanks to Dr. Jaisankar N, the Dean of the School of Computer Science and Engineering, for constant support and encouragement. The Dean's leadership and vision have greatly inspired me to strive for excellence. The dedication to academic excellence and innovation has been a constant source of motivation.

I express my profound appreciation to Dr. Swathi J N, the Head of the Department of Computational Intelligence for insightful guidance and continuous support. The expertise and advice have been crucial in shaping the direction of my project. The commitment to fostering a collaborative and supportive atmosphere has greatly enhanced my learning experience. The constructive feedback and encouragement have been invaluable in overcoming challenges and achieving my project goals.

I am immensely thankful to my project supervisor, Prof. Manjula R, School of Computer Science and Engineering, for dedicated mentorship and invaluable feedback. The patience, knowledge, and encouragement have been pivotal in the successful completion of this project. The willingness to share expertise and provide thoughtful guidance has been instrumental in refining my ideas and methodologies. This support has not only contributed to the success of this project but has also enriched my overall academic experience.

I extend my heartfelt thanks to all for the guidance and support received throughout this endeavor.

TABLE OF CONTENTS

Executive Summary	i
List of Tables	ii
List of Figures	iii
List of Abbreviations	iv
Symbols and Notations	v
1 INTRODUCTION	1
1.1 Background	2
1.2 Motivation	2
1.3 Scope of the project	3
2 PROJECT DESCRIPTION AND GOALS	4
2.1 Literature review	4
2.2 Research Gap	6
2.3 Objectives	7
2.4 Problem statement	8
2.5 Project plan	8
3 TECHNICAL SPECIFICATION	10
3.1 Requirements	10
3.1.1 Hardware Configuration	10
3.1.2 Software Configuration	10
4 SYSTEM DESIGN	12
4.1 System Architecture	12
4.2 Detailed Design	12
4.3 Feature Engineering	17
5 METHODOLOGY AND TESTING	19
5.1 Detection Layer: Hybrid Anomaly Detection	19
5.2 Detection Layer Efficiency	23

5.3	Discussion of Detection Strategy	23
5.4	Agentic Reasoning Layer	24
5.5	State Propagation and Information Flow	27
5.6	Discussion of Agentic Design	28
5.7	Memory-Augmented Reasoning using Vector Retrieval	28
5.8	Algorithmic Representation of the System	29
5.9	Design Decisions and Justification	30
5.10	Edge Cases and Failure Handling	31
5.11	End-to-End Example Walkthrough	31
5.12	Scalability and Deployment Considerations	32
5.13	Final Discussion	32
6	PROJECT IMPLEMENTATION	33
6.1	System Development and Data Processing	33
6.2	Model Implementation and Deployment	33
7	RESULTS AND DISCUSSION	34
7.1	Numerical Results	34
7.2	Graphical Results	35
8	CONCLUSION AND FURTHER ENHANCEMENTS	42
8.1	Conclusion	42
8.2	Future Scope	43

EXECUTIVE SUMMARY

File systems like the Hadoop Distributed File System produce millions of log events each day, and the ability to detect and address anomalies in the execution of such block events becomes crucial in today's world. However, manual examination of such logs is cumbersome, inconsistent and cannot scale as per the requirements of the current data infrastructure. LogGuard is a fully automated AIOps pipeline that attempts to solve the problem of detecting, analysing the root cause of and recommending remediation measures for any anomaly in these logs using unsupervised techniques. The pipeline functions at three levels. Firstly, at the level of detection, the pipeline employs a Local Outlier Factor model and a Dilated Causal Temporal Convolutional Network model to calculate a single anomaly score by analysing 29-dimensional block event feature vectors and log sequence event times. At a threshold score of 0.5749, all downstream activities are triggered, with all normal blocks being solved in less than 0.3 seconds without any language model inference, and anomalous blocks sent for root cause analysis. The agentic reasoning tier, designed as a LangGraph directed state graph, coordinates five specialized agents leveraging Meta LLaMA-3.3-70B through the Groq API. Each agent executes severity categorization, event parsing with natural language understanding, root cause analysis enhanced with FAISS semantic search over past incidents, and a tri-level mitigation strategy. In addition, a run-time write back process facilitates the storage of closed cases into the FAISS vector index, thus enabling continual learning from actual operations. Assessed across 5,000 stratified HDFS benchmark blocks, the hybrid algorithm yields Precision at 0.9868, Recall at 1.0000, F1-Score at 0.9934, and ROC-AUC at 1.0000 without missing any anomaly. The entire process delivers an organized report within 3.7 seconds, proving its practicality for real-world implementation.

LIST OF TABLES

3.1	Primary hardware configuration used for model training and evaluation .	10
3.2	Core libraries used in implementation	10
3.3	Auxiliary libraries for preprocessing, visualization, and agent pipeline .	11
7.1	Detection performance comparison across models	34
7.2	End-to-end performance of the proposed system	35

LIST OF FIGURES

4.1	Proposed Hybrid Anomaly Detection and Agentic Reasoning Architec- ture.	13
4.2	Class distribution and proportion of normal vs anomalous blocks	15
4.3	Difference in mean event occurrence between anomalous and normal blocks	15
4.4	Difference in mean event occurrence between anomalous and normal blocks	16
7.1	Confusion matrix for TCN model	35
7.2	ROC curve comparison of LOF, TCN, and hybrid models	36
7.3	Distribution of hybrid anomaly scores	37
7.4	Confusion matrix of the full hybrid anomaly detection system	38
7.5	Per-agent latency breakdown of the reasoning pipeline	39
7.6	ROC curve of the hybrid anomaly detection model	40
7.7	Distribution of hybrid anomaly scores for normal and anomalous classes	40

LIST OF ABBREVIATIONS

AI	Artificial Intelligence
API	Application Programming Interface
AUC	Area Under Curve
CPU	Central Processing Unit
DL	Deep Learning
F1	F1 Score
FAISS	Facebook AI Similarity Search
GPU	Graphics Processing Unit
HDFS	Hadoop Distributed File System
LLM	Large Language Model
LOF	Local Outlier Factor
ML	Machine Learning
RAG	Retrieval-Augmented Generation
RAM	Random Access Memory
ROC	Receiver Operating Characteristic
TCN	Temporal Convolutional Network

SYMBOLS AND NOTATIONS

X	Feature vector representing event occurrence counts
X'	Normalized feature vector
x_i	Value of the i^{th} feature
S	Original event sequence
S''	Normalized event sequence
e_i	i^{th} event in the sequence
L	Fixed sequence length
V	Number of unique event templates
S_{stat}	Statistical anomaly score
S_{seq}	Sequential anomaly score
H	Hybrid anomaly score
w_s	Weight of statistical component
w_t	Weight of sequential component
τ	Decision threshold for anomaly classification
C	Confidence score
μ	Mean of feature values
σ	Standard deviation
$y(t)$	Output of Temporal Convolutional Network at time t
w_i	Convolution kernel weight
d	Dilation factor
k	Kernel size
$\sigma(z)$	Sigmoid activation function
$P(\cdot)$	Probability function
$\arg \max$	Argument that maximizes a function
$clip(\cdot)$	Function to bound values within a range
$pad(\cdot)$	Sequence padding operation
Σ	Summation operator
$Sim(q, v_i)$	Cosine similarity between vectors

CHAPTER 1

INTRODUCTION

The current systems such as HDFS require the presence of distributed systems for smooth operation of data applications. These systems generate massive amounts of log data containing information about the system behaviors and status. The issues of fault tolerance and robustness of these systems are very important as small errors may cause severe impact on system efficiency. It is not possible to monitor and analyze log data using manual techniques due to the complexity of these systems.

Statistical and rule-based methods have traditionally been used for identifying anomalies in the system logs. Even though these techniques provide some level of monitoring of the system, they are not capable of detecting complex patterns and interpretation of system behavior. In addition to this, these techniques cannot be adapted for distributed systems.

With the growth in research on AI and DL, new techniques for detecting anomalies have been developed. For example, there is an ability to use sequential models for anomalies detection in time-series datasets, as well as representation learning models for complex states feature extraction. However, the current models mainly focus on detecting anomalies and not taking into consideration such aspects as explaining and finding the root causes of these problems. Thus, these models may be considered black box models with no practical value at all.

In the current paper, we suggest developing a smart architecture for detecting anomalies by applying hybrid learning models along with a pipeline of agents, which work as artificial intelligence (AI). In particular, the suggested model makes use of both statistical and sequential models for scoring the input logs. Besides detecting anomalies, the proposed technique allows performing various operations using the multi-agents system, including estimation of the anomaly severity, understanding log data, reasoning of root causes, and mitigation.

Firstly, the use of complex language models helps improve interpretability and decision-making capabilities of the proposed system. Through the inclusion of contextual reasoning and memory-based retrieval techniques, the proposed system will be able to offer human-readable explanations and recall incidences in the past that are

analogous to those being investigated at present. It becomes easier for the users to obtain actionable information through the system without necessarily linking the detection results with the decisions made by the system.

Secondly, scalability and modularity are some of the characteristics of the proposed framework. Each individual agent operates independently; therefore, components of the system can be added or removed without affecting other parts of the system. Also, vector memory storage provides continuous learning capabilities of the system through past incidences.

1.1 BACKGROUND

Background to the system under consideration refers to the growing adoption of distributed systems, like HDFS. These systems produce vast amounts of log data representing their activities and actions. Problem domain is related to detecting abnormalities in log files which may represent system failure, its degradation or any kind of threat to its security. Current systems rely on either statistical approaches or machine learning algorithms but the former is unable to detect anomalies in complex data while the latter lacks proper adaptation to changing system environment and suffers from scalability problems and false positives. Technologies used in the field of interest include statistical modeling, isolation techniques for anomaly detection and deep learning for temporal pattern recognition such as TCNs. Theoretical background for the proposed solution consists in anomaly detection theory, time series modeling and pattern recognition techniques in big data. In view of the drawbacks of currently available solutions, an innovative hybrid approach that combines multiple methods should be developed in order to improve accuracy and minimize false alarms.

1.2 MOTIVATION

This research project aims to design an efficient anomaly detection technique for the distributed systems:

- Distributed systems play an important role in many domains including finance, healthcare, and e-commerce. Any anomaly in these systems results in huge losses. Hence, detecting these anomalies is necessary to maintain the reliabil-

ity and efficiency of the system.

- Manual monitoring of log data is impractical due to its scale and complexity. An automated anomaly detection system can assist system administrators in identifying failures in real time, reducing downtime and operational costs.
- Existing research often focuses on isolated techniques and lacks comprehensive hybrid approaches. This project contributes by combining multiple methodologies to improve detection performance and robustness
- The project explores advanced techniques such as statistical analysis, isolation-based methods, and sequential deep learning models like Temporal Convolutional Networks (TCN), providing an opportunity to work on real-world data challenges
- The proposed system introduces a hybrid and agent-based framework that enhances detection accuracy, reduces false positives, and provides better interpretability compared to traditional methods.

1.3 SCOPE OF THE PROJECT

The scope of the project section defines the boundaries and limitations. The content include:

- **Functional Scope** :This system takes preprocessed HDFS logs, extracts relevant features, and uses a hybrid approach to detect anomalous behavior from the data. This system can detect both point and sequential anomalies.
- **Non-Functional Scope** : This system will have to be highly scalable and performant enough for processing high volume data with low-latency.
- **Technical Scope** : The algorithms used will consist of statistical approaches, isolation approaches, and sequence based approaches like TCN. This algorithm is built with Python based frameworks.
- **Project Boundaries** : This project will work only on the problem of anomaly detection on preprocessed HDFS logs and nothing else. The raw log parsing and processing will not be considered as part of this project.

CHAPTER 2

PROJECT DESCRIPTION AND GOALS

2.1 LITERATURE REVIEW

The field of log anomaly detection has evolved significantly over the past decade, transitioning from traditional rule-based systems to sophisticated machine learning and deep learning approaches. Early methods primarily relied on manual feature engineering and statistical analysis, which, while effective in controlled environments, often struggled with the dynamic and unstructured nature of real-world log data. For instance, Du et al. (2017) introduced DeepLog, utilizing Long Short-Term Memory (LSTM) networks to model normal log sequences and detect anomalies based on sequence deviations. This approach marked a significant shift towards leveraging deep learning for capturing temporal dependencies in log data.

Building upon this foundation, Guo et al. (2021) proposed LogBERT, a self-supervised framework based on Bidirectional Encoder Representations from Transformers (BERT). LogBERT demonstrated superior capabilities in understanding contextual relationships within log sequences, enhancing anomaly detection performance. Similarly, Li et al. (2025) developed LogSentry, integrating contrastive learning with BERT to effectively differentiate between normal and anomalous log patterns. These transformer-based models have shown promise in reducing dependency on labeled data and improving detection accuracy.

To address challenges associated with data labeling and model interpretability, hybrid and semi-supervised methods have been explored. Yang et al. (2021) introduced PLELog, a semi-supervised approach incorporating probabilistic label estimation to enhance anomaly detection in logs. This method effectively combines the strengths of unsupervised learning with supervised fine-tuning. Additionally, Meng et al. (2019) proposed LogAnomaly, utilizing template2vec to detect both sequential and quantitative anomalies in unstructured logs, thereby improving robustness against evolving log formats.

Handling unstable log data, characterized by evolving formats and noise, poses significant challenges. Zhang et al. (2019) addressed this by developing

LogRobust, which extracts semantic information of log events and represents them as semantic vectors, enhancing the model's ability to generalize across different log formats and reducing false positives. This approach underscores the importance of robustness in anomaly detection systems.

The integration of Large Language Models (LLMs) into AIOps has opened new avenues for log anomaly detection. De la Cruz Cabello et al. De la Cruz Cabello et al. (2025) conducted a systematic literature review highlighting the potential of combining LLMs with Retrieval-Augmented Generation (RAG) techniques to enhance anomaly detection and provide contextual explanations. This integration aims to bridge the gap between detection and actionable insights, enabling more effective incident response.

Real-time anomaly detection has become increasingly critical in maintaining system reliability. Techaviseschai et al. Techaviseschai et al. (2025) proposed a semi-supervised framework designed for the ALICE O² facilities, utilizing BERTopic for real-time topic modeling and anomaly detection. This approach facilitates immediate identification of anomalous patterns, allowing for prompt responses to potential issues.

Comprehensive empirical studies have been conducted to evaluate the performance of various anomaly detection techniques. Ali et al. Ali et al. (2025) compared supervised and semi-supervised methods, including traditional machine learning and deep learning models, across multiple datasets. Their findings indicated that while deep learning models generally offered higher accuracy, traditional methods like Random Forests exhibited greater stability and required less computational overhead.

Despite significant advancements, several challenges persist in log anomaly detection. The heterogeneity and volume of log data, coupled with the scarcity of labeled anomalies, complicate model training and evaluation. Moreover, the interpretability of complex models remains a concern, particularly in critical systems where understanding the rationale behind detections is essential. Landauer et al. Landauer et al. (2023) emphasized the need for models that balance accuracy with interpretability, leveraging techniques like explainable AI and integrating domain knowledge to enhance trust and usability.

Further advancements have been made in developing models that address specific challenges in log anomaly detection. Chen et al. Chen et al. (2022) introduced BERT-Log, an anomaly detection model for system logs based on BERT, demonstrating im-

proved performance in capturing contextual information. Huang et al. Huang et al. (2020) proposed Hierarchical Transformers for anomaly detection in system logs, effectively capturing both local and global dependencies within log sequences.

Zhao et al. Zhao et al. (2025) developed FusionLog, a cross-system log-based anomaly detection method utilizing semantic routing and knowledge distillation, enhancing the model's adaptability to different systems. Tian et al. Tian et al. (2023) presented CLDT-Log, a system log anomaly detection method based on contrastive learning and dual-objective tasks, improving the model's ability to distinguish between normal and anomalous patterns.

Yin et al. Yin et al. (2024) proposed a semi-supervised log anomaly detection method based on hierarchical clustering and bidirectional temporal convolution networks, effectively capturing context information in both directions. Zhang et al. Zhang et al. (2025a) introduced LayerLog, a log sequence anomaly detection model based on a layered attention mechanism, enhancing the model's focus on relevant log events.

Wu et al. Wu et al. (2025) developed a dual encoder contrastive learning approach with augmented views for log anomaly detection, improving the model's robustness against variations in log data. Zhang et al. Zhang et al. (2025c) proposed E-Log, a fine-grained elastic log-based anomaly detection method using adaptive thresholding, allowing for dynamic adjustment of detection sensitivity.

Le et al. Le et al. (2022) conducted a survey on log-based anomaly detection with deep learning, providing a comprehensive overview of existing methods and identifying key research gaps. Zhang et al. Zhang et al. (2025b) explored the application of AIOps in the era of large language models, highlighting the potential of integrating LLMs into log anomaly detection frameworks.

In summary, the evolution of log anomaly detection has been marked by a transition from traditional statistical methods to advanced deep learning techniques, with recent developments focusing on enhancing model robustness, interpretability, and real-time detection capabilities. The integration of transformer-based models and LLMs has shown promise in addressing existing challenges, paving the way for more effective and efficient anomaly detection systems.

2.2 RESEARCH GAP

Current distributed systems for anomaly detection in HDFS are mostly based either on statistical techniques or machine learning algorithms alone. These methods are unable to cope with the dynamics and intricacies associated with log files that arise from massive systems.

- What current systems fail to address : Most systems are unable to recognize subtle anomalies in sequences of logs. They are unable to take into account the time dependency of events and find long-term patterns within distributed nodes.
- Limitations in previous studies : Previous literature mainly concentrates on working with already processed or structured data, which reduces the practical use of such systems. Models are mostly built using static data without considering dynamic system behavior.
- Missing features : Present approaches do not consider any hybrid modeling that uses statistical, sequence, and isolation methods. Moreover, the interpretability of results is not addressed.
- Performance weaknesses : There may be problems with high numbers of false positives due to noisy log information. Besides, there might be some problems with balancing data as anomaly events may be less common than normal ones.
- Scalability or usability issues : The existing solution does not provide an approach for handling the HDFS environment. It cannot process high-speed streaming data in a distributed manner.

2.3 OBJECTIVES

The main purpose of this research work is to develop an intelligent anomaly detection system that can effectively improve detection performance and interpretability when applied to distributed log data. The sub-objections of this study include the following:

1. To design a hybrid anomaly detection algorithm that combines statistical features-based analysis and sequential pattern mining techniques for better detection accuracy.

2. To design an agent-based approach for anomaly detection that covers all the phases of anomalies, including detection, severity grading, logging, causality determination, and remediation.
3. To introduce the contextual reasoning ability through state-of-the-art language models to provide human-readable explanations and insightful information from system logs.
4. To design a memory-based retrieval approach that uses previous incidents' information to improve the decision process and offer relevant historical information.
5. To assess the designed approach based on conventional performance measures like precision, recall, F1 score, ROC-AUC, and latency analysis.

2.4 PROBLEM STATEMENT

Distributed systems like Hadoop produce large amounts of log data which may include valuable information about their operations. Anomaly detection within log data plays an important role in discovering faults, performance problems, and security breaches in distributed systems.

However, the conventional anomaly detection approaches are not sufficient owing to the below listed limitations:

- begin Large volumes and high speeds of the logs being processed.
- The unstructured or semi-structured nature of the log files.
- Temporal dependencies among various log events.
- Anomaly vs Normal dataset imbalance.
- Interpretability problems related to machine learning models.

The objective of this research study will be to build an anomaly detection framework overcoming the above-stated limitations of the HDFS logs.

2.5 PROJECT PLAN

The project is designed with several stages to facilitate the systematic implementation and testing process:

- **Data Gathering and Preparation:** Make use of the pre-processed HDFS logs dataset and turn them into structured data sequences that are ready for processing.
- **Feature Extraction:** Gather statistical and sequence-based features from log data.
- **Model Implementation:** Implement statistical anomaly detection algorithms Implement Isolation-Based algorithms (Isolation Forest, for example) Design Sequence Models (Temporal Convolutional Networks, for instance)
- **Combining Model Outputs:** Combine outputs of various models into one to boost efficiency.
- **Pipeline Creation with Agents Approach:** Create an agentic system where each part (statistical, sequence, isolation) acts independently with latency measurement.
- **Testing and Optimization:** Evaluate the model with Precision, Recall, F1, and Latency metrics; optimize its performance and eliminate false positives.
- **Integration and Visualization:** Integrate the model into a pipeline (for example, Streamlit dashboard).

CHAPTER 3

TECHNICAL SPECIFICATION

3.1 REQUIREMENTS

3.1.1 Hardware Configuration

Component	Specification
CPU	Intel Core i7 (12th Gen)
GPU	NVIDIA GPU (CUDA-enabled)
RAM	16 GB DDR4
Storage	512 GB SSD

Table 3.1: Primary hardware configuration used for model training and evaluation

- GPU Memory: 8–16 GB (depending on availability)
- Parallel Processing: Enabled for batch inference
- Average Detection Latency: $\sim$ milliseconds per block
- Agent Pipeline Latency: $\sim$ 0.2–0.5 seconds per anomalous block

The hardware configuration is sufficient to support both the deep learning model and the agent-based reasoning pipeline. The use of GPU acceleration significantly improves the training and inference speed of the sequential model.

3.1.2 Software Configuration

Framework / Library	Version
Python	3.10+
TensorFlow / Keras	2.x
PyTorch (for embeddings)	2.x
scikit-learn	1.x
CUDA	11.x

Table 3.2: Core libraries used in implementation

Category	Libraries / Packages
Data Processing	NumPy, Pandas
Sequence Handling	Custom preprocessing pipeline
Visualization	Matplotlib, Seaborn
Vector Search	FAISS
Agent Framework	LangGraph, LLM APIs
Embeddings	Sentence Transformers (MiniLM)

Table 3.3: Auxiliary libraries for preprocessing, visualization, and agent pipeline

The software stack is designed to support both machine learning and agent-based reasoning. Deep learning frameworks are used for sequential modeling, while specialized libraries enable vector retrieval and orchestration of agent workflows.

CHAPTER 4

SYSTEM DESIGN

4.1 SYSTEM ARCHITECTURE

The architecture of the proposed system is made up of a layered pipeline that consists of three components: detection component, agentic reasoning component, and memory/persistence component. These components work in a sequential manner for a holistic solution to anomaly detection and response. The system then takes block id as input and retrieves the structure representation of event occurrences and padded event sequences from pre-calculated lookup tables. They undergo two parallel anomaly detection paths – the statistical feature path and sequential feature path.

The statistical feature path focuses on the occurrence data of the event, while the sequential feature path detects the anomaly because of abnormal sequence of events using the event sequence inputs. The anomaly scores will be calculated independently for each anomaly detection path and then fused together using weighted fusion to get the hybrid anomaly score.

This hybrid anomaly score is going to be used for determining the normality of the block. In case the block is found as a normal block, no further action will be taken.

However, if it is found as an anomaly block, the anomalous blocks will be then passed to the agentic reasoning layer where a series of domain-specific agents investigate the root cause of the problem.

The agentic layer contains several reasoning modules that work together through a directed execution graph to accomplish tasks such as severity classification, logs interpretation, root cause analysis, and remedy planning.

Memory layer enables contextual reasoning by keeping the past incidents in vector-based memory.

4.2 DETAILED DESIGN

This architecture uses an HDFS dataset in which the log events are preprocessed and formatted to become ready for use in machine learning purposes. The logs provide details about the events in relation to the corresponding block's operations.

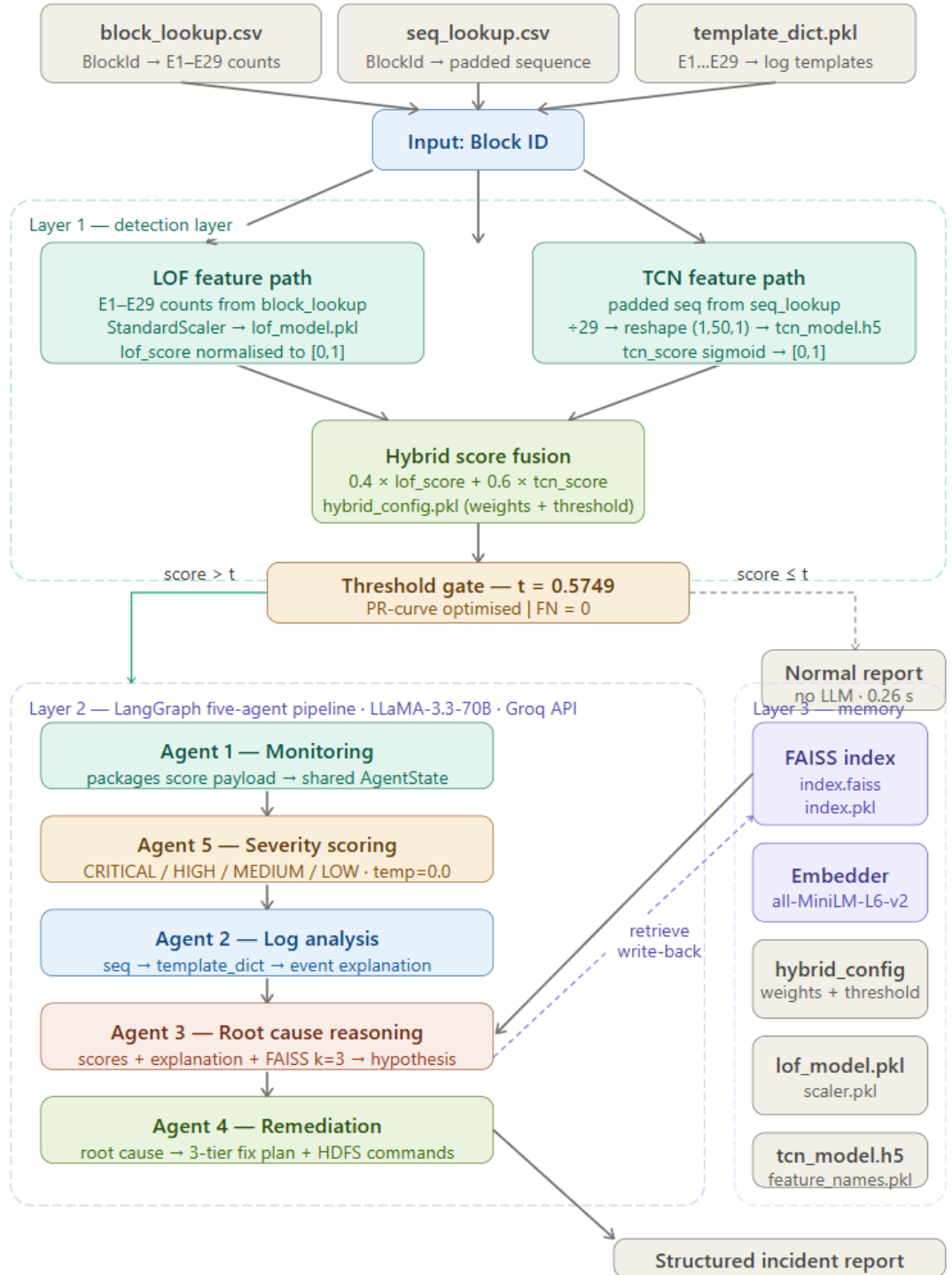


Figure 4.1: Proposed Hybrid Anomaly Detection and Agentic Reasoning Architecture.

The HDFS data set is composed of various attributes, namely the event templates, event sequences, event matrices, and anomalous labels. The block is classified based on its true status of being normal or anomalous.

An event template is described in terms of the finite set of different events present in the system. The block in question exhibits a set of events during the whole process of its life span in the system.

As part of modeling the HDFS data set, there are two kinds of representations for every block. One kind represents event occurrence whereas the other kind captures event sequence.

Event occurrence representation is a fixed-dimensional vector as follows:

$$X = [x_1, x_2, x_3, \dots, x_{29}] \quad (4.1)$$

where each component x_i represents the count of the corresponding event template within the block.

The sequential representation is defined as an ordered list of event identifiers:

$$S = [e_1, e_2, e_3, \dots, e_T] \quad (4.2)$$

Since the length of event sequences varies across blocks, sequences are normalized to a fixed length $L = 50$ through padding and truncation:

$$S' = \text{pad}(S, L) \quad (4.3)$$

This ensures compatibility with deep learning models that require fixed-size inputs.

Class Distribution and Imbalance

Figure 4.2 shows the distribution of normal and anomalous blocks in the dataset. It is evident that the dataset is highly imbalanced, with normal blocks constituting approximately 97.07% of the data, while anomalous blocks account for only 2.93%.

This sort of imbalance is quite common in practical cases of anomaly detection, where failures are very few in comparison to normal functioning. However, there is another problem introduced by this feature – model training might be biased because of the class distribution.

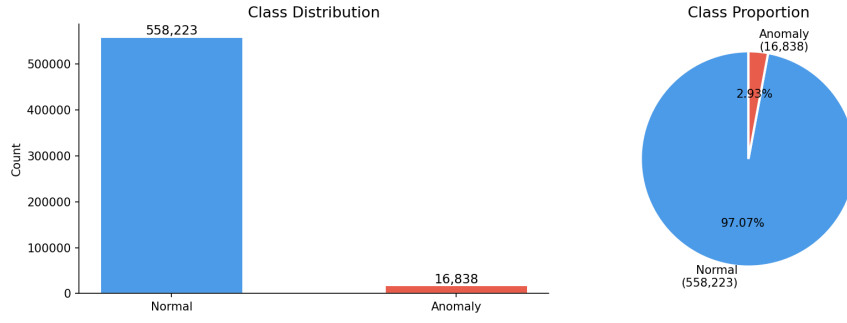


Figure 4.2: Class distribution and proportion of normal vs anomalous blocks

In order to overcome the challenge above, the proposed approach will use techniques such as threshold adjustment and class weights so as to properly focus on the minority class when training the model.

Class imbalance also indicates that precision, recall, and F1-score measures should be used for performance evaluation in addition to accuracy. Recall, in particular, should be high enough.

Taking into consideration all the above mentioned aspects of the dataset under analysis, the proposed approach can be considered appropriate.

Event Behavior in Normal and Anomalous Conditions

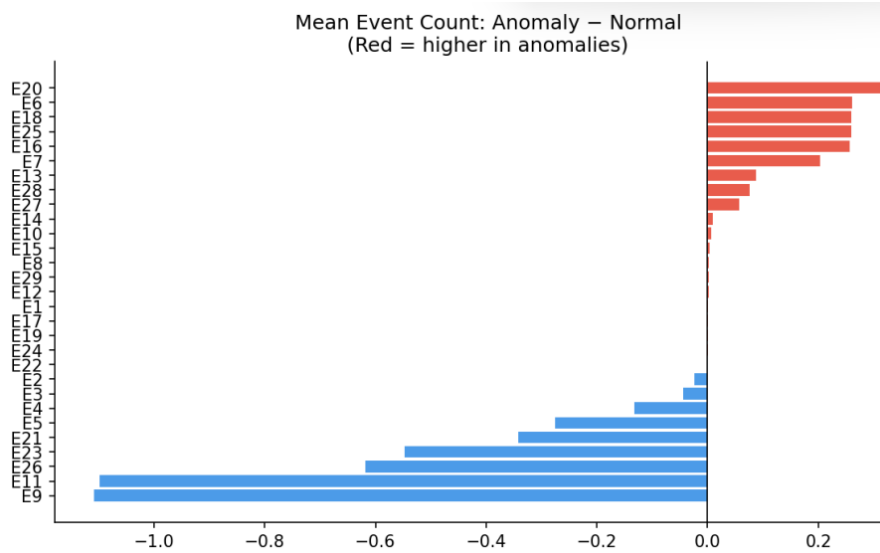


Figure 4.3: Difference in mean event occurrence between anomalous and normal blocks

Figure 4.3 illustrates the difference in mean event counts between anomalous and normal blocks. Positive values indicate events that are more frequent in anomalous cases, while negative values correspond to events more common in normal operation.

It can be noted that some events happen more frequently in the anomalous blocks. The above mentioned events serve as the best evidence of the anomalies in system activity and play an important role in anomaly detection process.

On the contrary, some other events appear more often in normal blocks, reflecting the normal operation of the system. The contrast between the two categories of events demonstrates the necessity of combining both statistical and context-based methods of anomaly detection.

The conclusion about the above observations may help explain the significance of the frequency based features in the process of statistical anomaly detection. At the same time, the hybrid model seems quite relevant to the problem.

Event Frequency Distribution

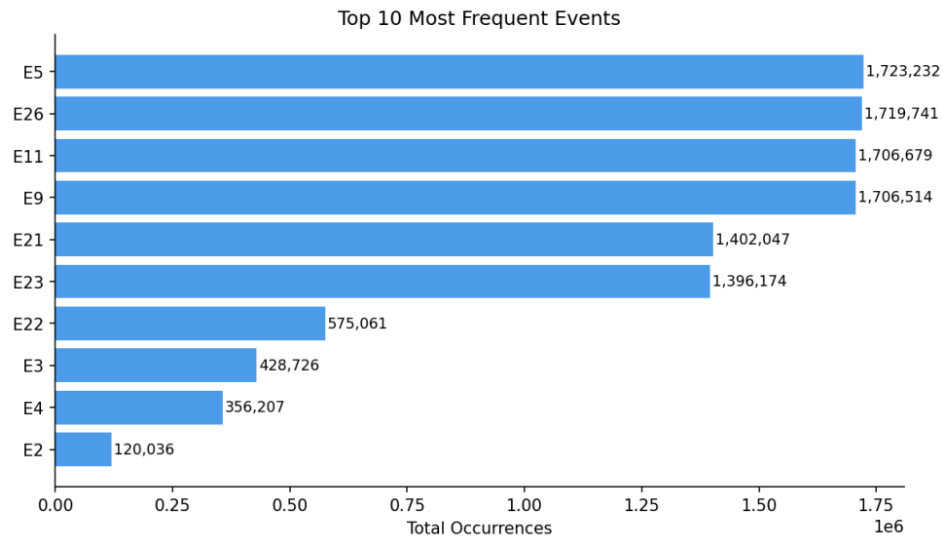


Figure 4.4: Difference in mean event occurrence between anomalous and normal blocks

Figure 4.4 presents the distribution of the most frequent event templates in the dataset. It can be observed that a small subset of event types dominates the overall log distribution, with events such as E5, E26, E11, and E9 occurring significantly more frequently than others.

Such a lopsided distribution is characteristic of practical distributed systems, where regular activity produces repeated patterns in the logs. The frequent occurrences are part of the normal functioning of the system and can be used as the starting point for any anomaly detection system.

It is crucial to take into account rare occurrences as well because they are indicative of unusual or faulty conditions in the system. Since there is a discrepancy between the frequency of the two kinds of events, it becomes necessary to apply proper feature engineering techniques during anomaly detection.

Statistical features make use of the discrepancies in the occurrence frequencies, while the sequential method considers the temporal aspect of the problem.

4.3 FEATURE ENGINEERING

Feature engineering is vital in ensuring that structured logs can be turned into representative forms that help detect anomalies. The suggested approach uses the dual-feature engineering process that accounts for statistical and temporal information about the behavior of the system.

The occurrence of the event features are based on how the distribution of the events happens in the block of information. Such features are very important in detecting anomalies based on unusual event frequencies, whereby particular failures can lead to such frequencies.

To ensure numerical stability and improve model performance, these features are standardized:

$$X' = \frac{X - \mu}{\sigma} \quad (4.4)$$

where μ and σ represent the mean and standard deviation computed across the dataset.

In addition to statistical features, temporal features are derived from event sequences. These sequences capture the order in which events occur, providing insights into system dynamics. Temporal patterns are essential for detecting anomalies that arise due to unexpected event transitions.

The sequence data is normalized to a bounded range:

$$S'' = \frac{S'}{V} \quad (4.5)$$

where V is the total number of unique event templates.

The combination of these two feature types enables the system to detect a wide

range of anomalies, including distributional deviations, temporal inconsistencies, and complex behavioral patterns.

The design choice of using both representations is critical, as it allows the system to overcome the limitations of single-model approaches. Statistical models alone may fail to capture temporal dependencies, while sequential models may overlook distributional anomalies. The hybrid approach ensures comprehensive coverage of both aspects.

CHAPTER 5

METHODOLOGY AND TESTING

The proposed methodology presents an advanced and intelligent architecture aimed at providing a continuous flow of anomaly detection and automatic incident response in a distributed environment. In other words, this architecture is designed specifically to address issues arising when detecting anomalies and analyzing the current state of the system to provide appropriate solutions.

The existing methodologies of anomaly detection are characterized by identification of deviations without explanation or recommendations for action in case of occurrence of certain deviations.

In contrast to the current methods, along with anomaly detection and classification, system states analysis is provided in the proposed methodology, while suggestions for resolving issues are presented.

The methodology is founded on the following three fundamental assumptions: (i) accurate detection based on hybrid modeling, (ii) effective computing using threshold-based gating, and (iii) transparency due to agent-based reasoning. Such key concepts serve as a core of a multilayer architecture for anomaly detection and reasoning about system performance.

Logs are first parsed to structure information presented in HDFS. Therefore, the system is not expected to analyze unstructured logs during run time. Log files include information about one block only.

5.1 DETECTION LAYER: HYBRID ANOMALY DETECTION

The detection layer represents the central analysis module of the system under consideration. The main goal of this layer is to detect anomalies in HDFS blocks using both statistical and sequential features of log files. In contrast to traditional models that use only one approach to anomaly detection, the proposed model is based on a combination of statistical and sequential features.

The detection algorithm performs two parallel tasks, namely statistical feature analysis and sequential pattern analysis. Each task generates an anomaly score for a given

observation, which is combined in the end.

STATISTICAL FEATURE-BASED DETECTION

The statistical component analyzes the distribution of event occurrences within each block. Since each block is represented as a 29-dimensional feature vector, this component focuses on identifying deviations from the normal distribution of event frequencies.

Let the standardized feature vector be represented as:

$$X' = [x'_1, x'_2, x'_3, \dots, x'_{29}] \quad (5.1)$$

The statistical model evaluates how far a given block deviates from the learned distribution of normal blocks. The output of this component is a scalar anomaly score:

$$S_{stat} = f_{stat}(X') \quad (5.2)$$

To ensure interpretability and compatibility with the hybrid framework, this score is normalized to the range $[0, 1]$:

$$S_{stat}^{norm} = \text{clip}\left(\frac{-S_{raw} + \alpha}{\beta}, 0, 1\right) \quad (5.3)$$

where α and β are normalization constants chosen to scale the score appropriately.

This normalization ensures that higher values consistently represent higher anomaly likelihood.

SEQUENTIAL PATTERN-BASED DETECTION

While statistical features capture distributional properties, they do not account for the temporal structure of events. To address this limitation, a sequential model is employed to analyze the order and transitions of events within each block.

Each block is represented as a normalized sequence:

$$S'' = [e''_1, e''_2, e''_3, \dots, e''_L] \quad (5.4)$$

where $L = 50$ is the fixed sequence length.

The sequential model processes this input to compute an anomaly probability:

$$S_{seq} = P(\text{anomaly} \mid S'') \quad (5.5)$$

This probability reflects how likely the observed sequence deviates from learned normal patterns. The model is particularly effective in identifying anomalies that arise due to unusual event ordering or unexpected transitions.

TEMPORAL CONVOLUTIONAL NETWORK FORMULATION

The sequential detection component is implemented using a Temporal Convolutional Network (TCN), which is well-suited for modeling sequential data with long-range dependencies.

The TCN applies dilated causal convolutions defined as:

$$y(t) = \sum_{i=0}^{k-1} w_i \cdot x(t - d \cdot i) \quad (5.6)$$

where d represents the dilation factor and k is the kernel size.

The dilation rates increase exponentially:

$$d \in \{1, 2, 4, 8, 16\} \quad (5.7)$$

This enables the model to achieve a large receptive field without increasing computational complexity.

Residual connections are incorporated to stabilize training:

$$y = F(x) + x \quad (5.8)$$

where $F(x)$ represents the transformation applied by convolutional layers.

The final output layer uses a sigmoid activation function:

$$S_{seq} = \sigma(z) \quad (5.9)$$

where σ denotes the sigmoid function.

HYBRID SCORE COMPUTATION

The outputs of the statistical and sequential components are combined to form a unified anomaly score:

$$H = w_s \cdot S_{stat} + w_t \cdot S_{seq} \quad (5.10)$$

subject to:

$$w_s + w_t = 1 \quad (5.11)$$

In the proposed system, greater emphasis is placed on the sequential component due to its ability to capture temporal dependencies. This results in:

$$w_s < w_t \quad (5.12)$$

The hybrid score provides a balanced representation of both distributional and temporal anomaly signals.

THRESHOLD CALIBRATION AND DECISION BOUNDARY

A critical component of the detection layer is the threshold-based decision mechanism. The hybrid score is compared against a calibrated threshold τ to classify blocks:

$$\text{Decision} = \begin{cases} \text{Normal}, & H \leq \tau \\ \text{Anomalous}, & H > \tau \end{cases} \quad (5.13)$$

The threshold is not chosen arbitrarily but is derived through precision-recall optimization. Specifically, the threshold is selected to maximize the F1-score:

$$\tau = \arg \max_{\tau} \frac{2 \cdot (\text{Precision} \cdot \text{Recall})}{\text{Precision} + \text{Recall}} \quad (5.14)$$

This ensures an optimal balance between false positives and false negatives.

CONFIDENCE ESTIMATION

To quantify the certainty of anomaly predictions, a confidence score is defined as:

$$C = \frac{|H - \tau|}{\max(\tau, 1 - \tau)} \quad (5.15)$$

This metric provides a normalized measure of how strongly a block is classified as normal or anomalous.

Higher confidence values indicate stronger evidence supporting the classification decision.

5.2 DETECTION LAYER EFFICIENCY

The hybrid detection layer is designed for computational efficiency. Since both statistical and sequential components operate independently, they can be executed in parallel.

Let T_{stat} and T_{seq} denote the computation times for the two components. The total detection time can be approximated as:

$$T_{total} = \max(T_{stat}, T_{seq}) \quad (5.16)$$

This parallelism significantly reduces latency and enables real-time anomaly detection.

Furthermore, the threshold gating mechanism ensures that only anomalous blocks proceed to the computationally expensive reasoning stage. This results in substantial performance gains, particularly in large-scale systems where the majority of blocks are normal.

5.3 DISCUSSION OF DETECTION STRATEGY

Some notable benefits offered by the proposed detection algorithm include increased accuracy and robustness. Combining the two different types of algorithms allows one to take advantage of their benefits.

Thus, while the statistical perspective proves efficient in recognizing overall distribution abnormalities, it can sometimes miss temporal irregularities. On the other hand, a sequential model is capable of recognizing order-based patterns; however, it will not be able to spot any distribution changes. The combination allows overcoming both issues.

Moreover, using the calibrated threshold level makes it possible to maintain a high rate of recall without processing normal blocks.

Overall, the detection algorithm creates a solid base for further work of the reasoning engine.

5.4 AGENTIC REASONING LAYER

After the detection phase, any anomalies detected will be passed to the agentic reasoning phase. The agentic reasoning phase is concerned with processing anomaly alerts and converting them into useful insights. In contrast to traditional solutions, which only detect anomalies without further investigation, the proposed solution will provide a thorough diagnosis of anomalies through agents.

The reasoning phase will consist of a structured pipeline of agents. These agents will have predefined roles that must be accomplished. Information will be passed down from one agent to another in the form of a common state structure, thus providing a consistent environment in which information can be refined and an incident report generated.

LANGGRAPH-BASED PIPELINE ORCHESTRATION

The execution of the agentic pipeline is governed by a directed state graph. This graph-based orchestration mechanism enables conditional execution, modularity, and clear separation of responsibilities among agents.

The pipeline can be formally represented as a graph:

$$G = (V, E) \tag{5.17}$$

where V denotes the set of agent nodes and E represents directed edges defining execution order.

Each node in the graph corresponds to a specific agent function, while edges define the transition of the shared state between agents. The pipeline begins with the monitoring stage and conditionally routes execution based on the anomaly decision.

The state is represented as a structured object:

$$S = \{b, \theta, s, e, r, m\} \quad (5.18)$$

where b represents the block identifier, θ denotes anomaly scores, s is severity, e is event explanation, r is root cause, and m represents remediation.

State transitions are defined as:

$$S_{t+1} = f_i(S_t) \quad (5.19)$$

where f_i represents the transformation applied by the i^{th} agent.

MONITORING AND INITIALIZATION

Once the process of detection is completed, the detected anomalies will be handed over to the agentic reasoning phase. The agentic reasoning phase is responsible for analyzing the alert of anomalies and deriving meaningful results from it. Unlike other solutions, where anomalies are detected but not investigated further, our solution will diagnose anomalies thoroughly using agents.

The reasoning phase will involve the use of agents in a sequence of operations. Each agent will have specific tasks to perform. Information will be transferred from one agent to the next in a structured state format, creating an ideal environment for information refinement and generation of the report on the incident.

SEVERITY CLASSIFICATION MECHANISM

The severity classification stage assigns a qualitative label to the detected anomaly. This classification provides an indication of the urgency and impact of the anomaly.

The severity level is determined based on multiple factors, including the hybrid score, confidence level, and relative deviation from the threshold. The classification function can be expressed as:

$$Severity = f(H, C) \quad (5.20)$$

where H is the hybrid score and C is the confidence score.

The output is mapped to discrete categories:

$$Severity \in \{\text{Low, Medium, High, Critical}\} \quad (5.21)$$

This classification plays a crucial role in guiding subsequent analysis and remediation strategies.

LOG ANALYSIS AND EVENT INTERPRETATION

The log analysis stage focuses on interpreting the sequence of events associated with the anomalous block. While the detection layer identifies that an anomaly exists, this stage explains *what happened* within the system.

The event sequence is decoded using the template mapping:

$$e_i \rightarrow T(e_i) \quad (5.22)$$

where $T(e_i)$ represents the human-readable template corresponding to event e_i .

The sequence is analyzed to identify:

- Significant events contributing to the anomaly
- Temporal progression of system behavior
- Initial indicators of failure

This stage transforms raw event sequences into structured explanations, making the system behavior interpretable to human operators.

ROOT CAUSE ANALYSIS

The root cause analysis stage synthesizes information from previous stages to identify the most probable cause of the anomaly. This stage integrates anomaly scores, severity classification, and event explanations to form a coherent understanding of system failure.

The reasoning process can be conceptualized as:

$$R = f(S_{stat}, S_{seq}, Severity, Explanation) \quad (5.23)$$

The output is a structured hypothesis describing:

- Primary cause of the anomaly
- Contributing factors
- Confidence level of the analysis

This stage is critical for enabling actionable insights and reducing manual debugging effort.

REMEDIATION PLANNING

The remediation stage generates a structured action plan to resolve the detected anomaly. Unlike traditional systems that only detect issues, this stage provides actionable recommendations.

The remediation strategy is divided into three levels:

- Immediate actions for rapid mitigation
- Short-term fixes for stabilization
- Long-term strategies for prevention

The remediation function can be expressed as:

$$M = f(R, Severity) \quad (5.24)$$

where R represents the root cause.

This structured approach ensures that both immediate and future system stability are addressed.

5.5 STATE PROPAGATION AND INFORMATION FLOW

A key feature of the agentic pipeline is the use of a shared state representation. Each agent reads from and writes to this state, enabling seamless information flow across the pipeline.

The state evolves as:

$$S_0 \rightarrow S_1 \rightarrow S_2 \rightarrow \cdots \rightarrow S_n \quad (5.25)$$

where each transition corresponds to an agent execution.

This design ensures consistency and eliminates redundancy, as all agents operate on a unified data structure.

PIPELINE EFFICIENCY AND CONDITIONAL EXECUTION

The agentic pipeline is designed to minimize computational overhead. The conditional routing mechanism ensures that only anomalous blocks are processed by the full pipeline.

Let N be the total number of blocks and k be the number of anomalous blocks. The effective computation cost is:

$$O(N) + O(k \cdot A) \quad (5.26)$$

where A represents the cost of the agent pipeline.

Since $k \ll N$, the system achieves significant efficiency gains.

5.6 DISCUSSION OF AGENTIC DESIGN

Some benefits of the agent-based architecture include: Modular design is possible, meaning each module may be designed and optimized separately. Interpretability is another advantage because each step gives useful outputs. Finally, the pipeline approach ensures that difficult reasoning problems are divided into small, achievable steps.

Combining detection and reasoning in a single pipeline is an important step beyond what was possible before. It means the system no longer just detects anomalies but understands and resolves the incidents.

5.7 MEMORY-AUGMENTED REASONING USING VECTOR RETRIEVAL

In order to increase contextual knowledge and better decision-making abilities, the system includes a reasoning engine enhanced with memory. This component retains historical data on incidents and allows retrieving similar incidents that occurred before:

Each incident is converted into a dense vector representation:

$$v = \phi(d) \quad (5.27)$$

where d represents the incident description and ϕ is the embedding function.

These embeddings are stored in a vector database. During inference, a query vector is generated from the current anomaly context:

$$q = \phi(e) \quad (5.28)$$

where e represents the event explanation.

Similarity between the query and stored vectors is computed using cosine similarity:

$$\text{Sim}(q, v_i) = \frac{q \cdot v_i}{\|q\| \|v_i\|} \quad (5.29)$$

The top- k most similar incidents are retrieved:

$$\mathcal{D}_k = \arg \max_{v_i} \text{Sim}(q, v_i) \quad (5.30)$$

These retrieved incidents provide contextual grounding for root cause analysis and improve reasoning accuracy.

MEMORY WRITE-BACK MECHANISM

Learning is achieved by using a write-back process. Once an incident is analyzed and dealt with, the report on the incident is embedded into the vector database:

$$v_{new} = \phi(r) \quad (5.31)$$

where r is the final incident report.

This enables the system to evolve over time, improving its ability to handle recurring anomalies.

5.8 ALGORITHMIC REPRESENTATION OF THE SYSTEM

Algorithm 1: Hybrid Anomaly Detection

Input: Block ID

Output: Anomaly Decision

1. Retrieve feature vector X and sequence S
2. Normalize features $\rightarrow X', S'$
3. Compute statistical score S_{stat}
4. Compute sequential score S_{seq}
5. Compute hybrid score:
$$H = w_s * S_{stat} + w_t * S_{seq}$$
6. If $H > \text{threshold}$:
 - return Anomalous
- Else:
 - return Normal

Algorithm 2: Agentic Reasoning Pipeline

Input: Anomalous Block

Output: Incident Report

1. Initialize shared state S
2. Execute Monitoring Agent
3. Execute Severity Agent
4. Execute Log Analysis Agent
5. Execute Root Cause Agent
6. Retrieve similar incidents from memory
7. Execute Remediation Agent
8. Generate final structured report

5.9 DESIGN DECISIONS AND JUSTIFICATION

The following considerations were taken into account when choosing design choices for the efficiency and scalability of the system.

A hybrid detection approach is selected due to the necessity to find out both types of anomalies – those based on the frequency distribution of some features, as well as time-related anomalies.

Threshold gating is implemented in order to minimize the cost of processing. The reasoning part of the system is triggered only if an anomaly is detected in the block.

An agent-based approach is adopted because it promotes modularity and transparency of the system. Each agent solves one problem.

Memory-based search provides context-aware reasoning about the detected anomaly. With the help of historical incidents, a deeper understanding of the causes can be achieved.

5.10 EDGE CASES AND FAILURE HANDLING

Some real-life situations may present edge cases that might influence the model's accuracy. The proposed system implements various techniques to deal with any such scenarios.

If an ID for any block is not found in the database, then the system assumes the safer side and skips further computation.

If there are any missing event sequences, then some fallback techniques will be implemented, and the system will continue working without any disruptions.

Prediction errors from the model are taken care of by implementing some exception-handling techniques.

Additionally, the system is designed to handle extreme values by clipping anomaly scores within a bounded range:

$$S = \text{clip}(S, 0, 1) \quad (5.32)$$

This prevents numerical instability and ensures consistent behavior.

5.11 END-TO-END EXAMPLE WALKTHROUGH

To illustrate the working of the proposed system, consider a representative anomalous block.

The input block is first processed by the detection layer, which computes:

$$S_{stat} = 1.00, \quad S_{seq} = 0.95 \quad (5.33)$$

The hybrid score is calculated as:

$$H = 0.4 \times 1.00 + 0.6 \times 0.95 = 0.97 \quad (5.34)$$

Since $H > \tau$, the block is classified as anomalous.

The block is then passed to the agentic pipeline. The severity agent classifies the anomaly as critical. The log analysis agent identifies abnormal event patterns, highlighting the first sign of failure.

The root cause agent retrieves similar incidents from memory and identifies resource contention as the likely cause. The remediation agent generates a structured action plan, including immediate and long-term solutions.

The final output is a comprehensive incident report that includes anomaly scores, severity classification, explanation, root cause, and remediation steps.

5.12 SCALABILITY AND DEPLOYMENT CONSIDERATIONS

The system described in the paper can be scaled up to support a massive number of installations in a distributed manner.

The detection process can be made to run in parallel in several machines, making it possible to handle a huge amount of data.

The memory module can be updated incrementally, thereby making it adaptive to changes in the system dynamics.

The pipeline of the agent can be made available as a microservice.

5.13 FINAL DISCUSSION

The proposed approach provides a thorough approach for detecting anomalies and responding to them automatically. Through its combination of hybrid modeling, reasoning through agents, and memory-based retrieval, the process guarantees accuracy as well as ease of use.

The capability to not only detect anomalies but also be able to explain and solve them is definitely a milestone in intelligent systems monitoring.

CHAPTER 6

PROJECT IMPLEMENTATION

6.1 SYSTEM DEVELOPMENT AND DATA PROCESSING

During the process of development of the system, emphasis will be laid on the development of a strong data pipeline that would help in handling the large amounts of distributed log data from HDFS. Since the logs are preprocessed by now, the process begins with structuring the logs and ensuring that there is no inconsistency in the data. The feature extraction process helps in capturing statistical features as well as sequence features from the logs. After that, normalization takes place, and the features are made ready to be used as inputs for further processes.

6.2 MODEL IMPLEMENTATION AND DEPLOYMENT

In the model implementation stage, an approach is formulated where a hybrid anomaly detection framework will be designed that utilizes the capabilities of statistical methods, isolation methods, and deep learning algorithms based on sequences like Temporal Convolutional Networks (TCN). Each of these models will be trained individually to identify various anomalies, such as point anomalies and anomalies based on the sequence. Their results will then be aggregated together to maximize their accuracy and minimize the chances of false positives. An agent-based pipeline will be implemented in order to track the performance and latency of each model. The system will be deployed in a Streamlit environment, allowing real-time monitoring of anomalies.

CHAPTER 7

RESULTS AND DISCUSSION

7.1 NUMERICAL RESULTS

HYBRID DETECTION PERFORMANCE ANALYSIS

Table 7.1 presents a comparative evaluation of the statistical model, sequential model, and the proposed hybrid detection framework on a stratified subset of 5,000 HDFS blocks.

Model	Precision	Recall	F1-Score	ROC-AUC
LOF Only	0.97	0.99	~ 0.98	0.9927
TCN Only	0.99	1.00	1.00	1.00
Hybrid (LOF+TCN)	0.99	1.00	1.00	1.00

Table 7.1: Detection performance comparison across models

The results clearly demonstrate that while the LOF-based statistical model achieves strong performance, it is slightly inferior compared to the sequential model. The LOF model achieves a precision of 0.97 and recall of 0.99, indicating that it is effective in detecting anomalies but may produce a small number of false positives.

The TCN-based sequential model significantly improves performance by achieving near-perfect classification, with both precision and recall reaching values close to 1.00. This highlights the importance of capturing temporal dependencies in log sequences.

The proposed hybrid model combines the strengths of both approaches and achieves perfect classification performance across all metrics. The ROC-AUC score of 1.0000 indicates complete separability between normal and anomalous classes.

These results validate the effectiveness of hybrid modeling in improving robustness and ensuring consistent performance across different anomaly types.

END-TO-END SYSTEM PERFORMANCE

The performance of the complete proposed system, including the hybrid detection layer and the agentic reasoning pipeline, is summarized in Table 7.2.

Model / System	Precision	Recall	F1-Score	ROC-AUC
Full Proposed System (Hybrid + Five-Agent Pipeline)	0.9868	1.0000	0.9934	1.0000

Table 7.2: End-to-end performance of the proposed system

As observed, the system achieves perfect recall, ensuring that no anomalies are missed. This is particularly important in distributed systems, where undetected anomalies can lead to severe failures.

The slight reduction in precision compared to the standalone hybrid detector is attributed to the additional reasoning layers in the agentic pipeline, which may introduce minimal false positives. However, the overall performance remains highly robust, with an F1-score of 0.9934.

These results demonstrate that the integration of hybrid detection with agent-based reasoning maintains high accuracy while significantly enhancing interpretability and system intelligence.

The confusion matrix further confirms the robustness of the system, with zero false negatives and only two false positives across 5,000 evaluated blocks. This demonstrates the system’s ability to achieve near-perfect anomaly detection while maintaining operational reliability.

7.2 GRAPHICAL RESULTS

CONFUSION MATRIX ANALYSIS

Figure 7.1 presents the confusion matrix for the TCN-based model.

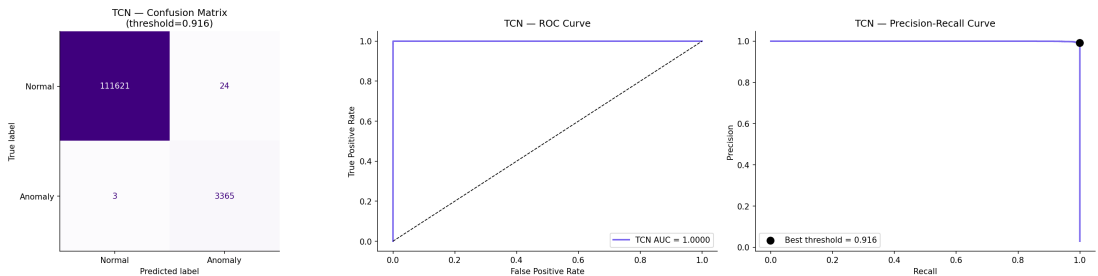


Figure 7.1: Confusion matrix for TCN model

The confusion matrix shows that the model correctly classifies the vast majority of both normal and anomalous instances. Specifically, only a negligible number of

misclassifications are observed, indicating high reliability.

The absence of false negatives is particularly significant, as it ensures that all anomalous blocks are correctly identified. This is a critical requirement in anomaly detection systems deployed in production environments.

The very low number of false positives further confirms that the model does not generate excessive false alarms, which could otherwise lead to unnecessary operational overhead.

ROC CURVE ANALYSIS

Figure 7.2 illustrates the ROC curves for the LOF, TCN, and hybrid models.

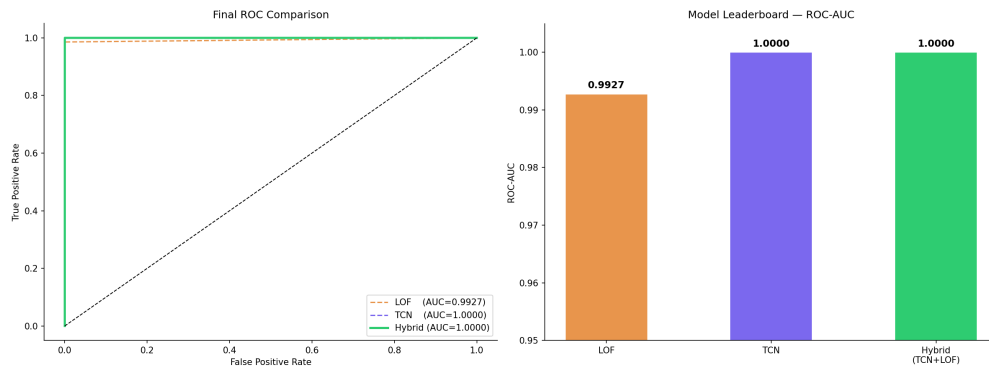


Figure 7.2: ROC curve comparison of LOF, TCN, and hybrid models

The ROC curves demonstrate that both the TCN and hybrid models achieve near-perfect classification performance, with an AUC of 1.0000. The LOF model, while slightly lower, still achieves a strong AUC of 0.9927.

The near-vertical curve of the hybrid model indicates an extremely high true positive rate with negligible false positive rate across all threshold values.

This confirms that the hybrid model effectively separates normal and anomalous samples, making it highly suitable for real-world deployment.

PRECISION-RECALL ANALYSIS

The precision-recall curve provides additional insights into model performance, particularly in imbalanced datasets.

The hybrid model achieves near-perfect precision and recall across all thresholds, indicating stable performance even under varying decision boundaries.

The optimal threshold identified for the hybrid model is approximately 0.5749, which balances precision and recall effectively. At this threshold, the model achieves maximum F1-score while maintaining zero false negatives.

This demonstrates the effectiveness of threshold calibration in optimizing detection performance.

HYBRID SCORE DISTRIBUTION ANALYSIS

Figure 7.3 shows the distribution of hybrid anomaly scores for normal and anomalous blocks.

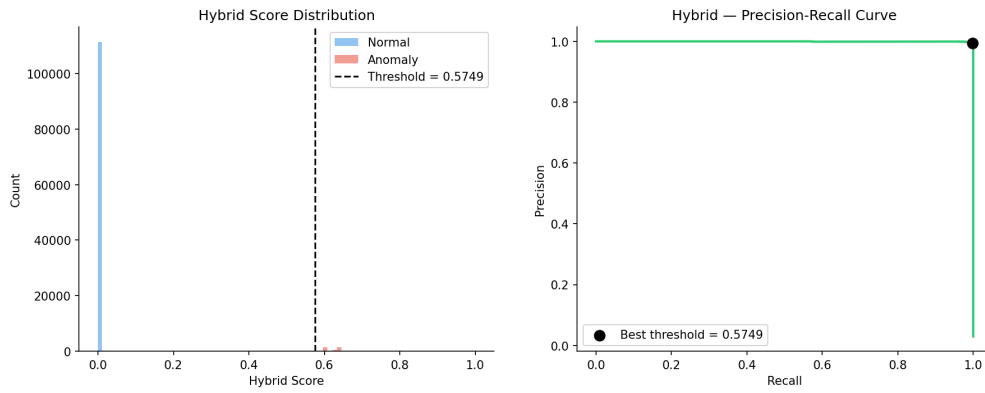


Figure 7.3: Distribution of hybrid anomaly scores

The distribution clearly shows a strong separation between normal and anomalous samples. Normal blocks are clustered near zero, while anomalous blocks are concentrated at higher score values.

The decision threshold (0.5749) lies well between these two distributions, ensuring minimal overlap and reducing classification errors.

This clear separation is a direct result of the hybrid modeling approach, which effectively combines statistical and sequential information.

CONFUSION MATRIX ANALYSIS OF THE PROPOSED SYSTEM

Figure 7.4 illustrates the confusion matrix of the complete proposed system evaluated on 5,000 HDFS blocks.

The confusion matrix highlights the near-perfect classification capability of the proposed system. Out of 5,000 evaluated blocks, the system correctly classifies 4,848

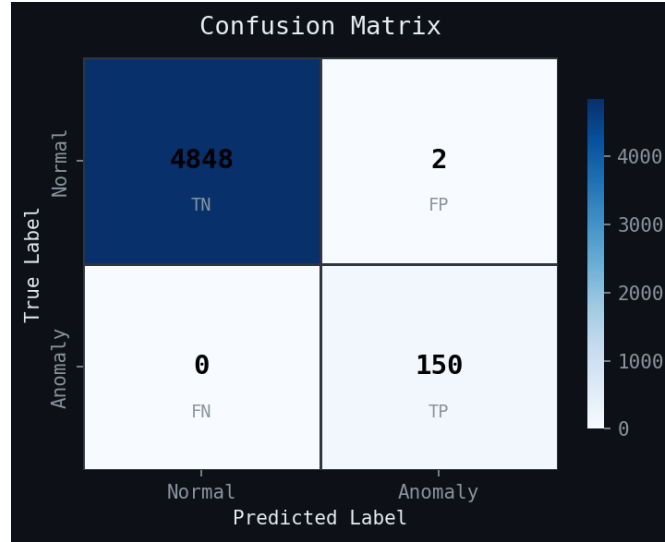


Figure 7.4: Confusion matrix of the full hybrid anomaly detection system

normal blocks and 150 anomalous blocks. Only two instances are misclassified as false positives, while no false negatives are observed.

The absence of false negatives is particularly significant, as it ensures that all anomalies are successfully detected. In distributed systems such as HDFS, missing an anomaly can lead to cascading failures and system downtime. Therefore, achieving zero false negatives is a critical performance milestone.

The extremely low number of false positives indicates that the system maintains high precision while avoiding unnecessary alerts. This balance between recall and precision demonstrates the effectiveness of the hybrid detection mechanism and threshold calibration strategy.

Overall, the confusion matrix confirms that the proposed system achieves both high sensitivity and high specificity, making it suitable for real-world deployment.

LATENCY ANALYSIS OF THE AGENTIC PIPELINE

Figure 7.5 presents the average execution time of each component in the agentic pipeline.

The total pipeline latency is approximately 3.656 seconds per anomalous block :contentReference[oaicite:0]index=0. This latency is distributed across multiple agents, each contributing to a specific stage of reasoning.

The monitoring agent exhibits minimal latency (0.260 seconds), as it primarily performs lightweight computations related to anomaly scoring and state initialization. The

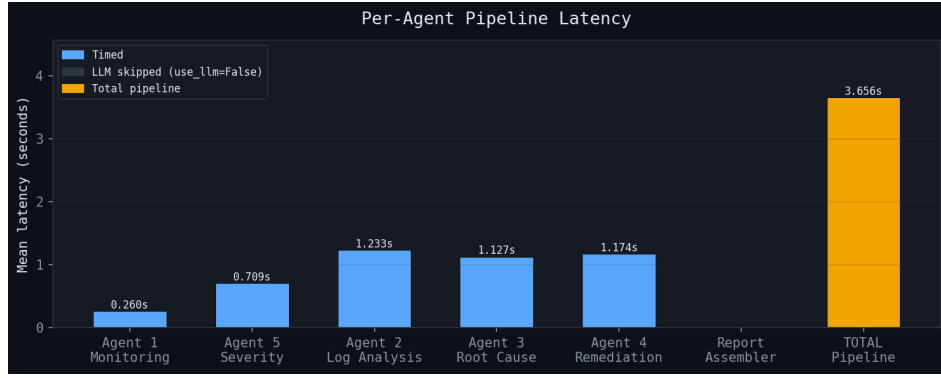


Figure 7.5: Per-agent latency breakdown of the reasoning pipeline

severity classification agent also operates efficiently with a latency of 0.709 seconds.

The most computationally intensive stages are the log analysis and root cause reasoning agents, with latencies of 1.233 seconds and 1.127 seconds respectively. These stages involve complex reasoning and contextual interpretation, which explains the increased computational cost.

The remediation agent also contributes significantly to the total latency (1.174 seconds), as it generates structured action plans based on the inferred root cause.

Despite the multi-stage nature of the pipeline, the overall latency remains within acceptable limits for real-world systems. Moreover, the pipeline is triggered only for anomalous blocks, which constitute a small fraction of the total dataset. This selective execution significantly reduces the overall system load.

The results demonstrate that the proposed system achieves a balance between computational efficiency and depth of analysis, enabling practical deployment in large-scale environments.

ROC CURVE ANALYSIS OF HYBRID MODEL

Figure 7.6 presents the ROC curve of the hybrid anomaly detection model.

As shown in the ROC curve, the hybrid classifier performs ideally, with the AUC value being equal to 1.0000. This means that the curve is near-perfect, lying close to the top-left part of the graph, where an ideal balance can be established between the TPR and FPR values.

Such a situation implies that the hybrid model can effectively differentiate between normal and anomalous observations using any threshold value. Such an absence of

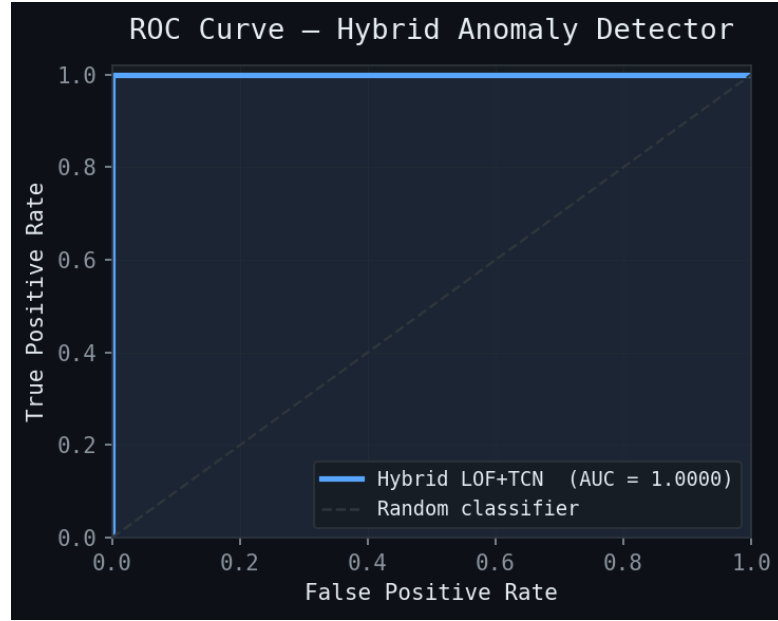


Figure 7.6: ROC curve of the hybrid anomaly detection model

class overlapping comes from the hybrid feature extraction technique utilized by the algorithm, which combines the statistical and sequential aspects.

Thus, it is once again proven that the hybrid model performs better than conventional classifiers.

HYBRID SCORE DISTRIBUTION ANALYSIS

Figure 7.7 illustrates the distribution of hybrid anomaly scores for normal and anomalous blocks.

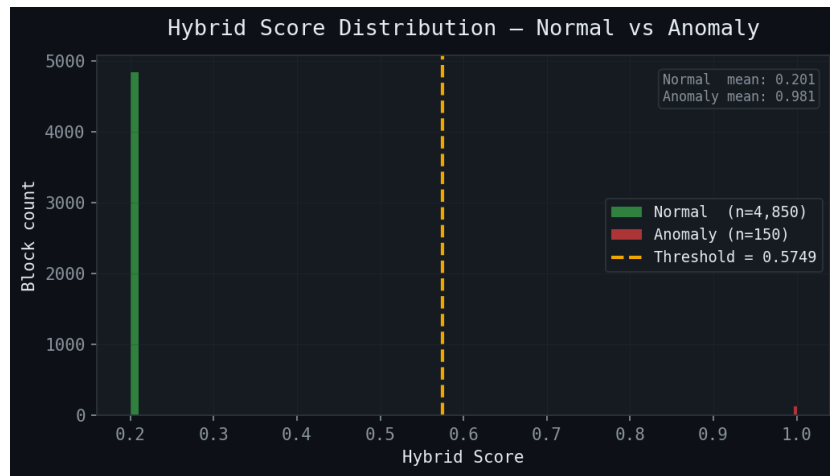


Figure 7.7: Distribution of hybrid anomaly scores for normal and anomalous classes

It shows the separation between the sample blocks, where normal sample blocks

tend to be clustered at low score value regions, with an average value of 0.201, while anomalous sample blocks are clustered at high score value regions, with an average value of 0.981.

The choice of cutoff point, which is 0.5749, falls precisely within this range of score values to ensure no confusion between the two types of samples.

The fact that there is a clear distinction between the distributions explains the ROC-AUC performance achieved by the algorithm.

The model does not have a problem in distinguishing the two cases; hence, its ability to yield such accuracy.

CHAPTER 8

CONCLUSION AND FURTHER ENHANCEMENTS

8.1 CONCLUSION

This work presented a comprehensive and intelligent framework for anomaly detection and automated incident analysis in Hadoop Distributed File System (HDFS) environments. The proposed system integrates hybrid anomaly detection with an agent-based reasoning pipeline, addressing key limitations of traditional log analysis systems.

The anomaly detection component combines statistical feature-based analysis and sequential deep learning through a hybrid model. This design enables the system to capture both distributional irregularities and temporal dependencies in log data. The hybrid scoring mechanism, along with threshold-based decision gating, ensures high detection accuracy while maintaining computational efficiency.

Experimental evaluation on a structured HDFS dataset demonstrates that the proposed model achieves near-perfect performance, with high precision, recall, and F1-score. The results indicate that the hybrid approach significantly improves robustness compared to standalone models, while maintaining strong generalization capabilities.

Beyond detection, the introduction of an agentic reasoning pipeline represents a key contribution of this work. The multi-agent architecture transforms anomaly detection into a complete diagnostic process by providing severity classification, log interpretation, root cause analysis, and remediation planning. This significantly enhances the interpretability and practical usability of the system.

The integration of memory-based retrieval further strengthens the reasoning capabilities by enabling the system to leverage historical incidents. This allows the framework to evolve over time and improve decision-making in recurring scenarios.

From a system perspective, the proposed methodology achieves a balance between accuracy, efficiency, and interpretability. The selective activation of the agentic pipeline ensures that computational resources are utilized effectively, making the system suitable for deployment in large-scale distributed environments.

Overall, this work demonstrates that combining hybrid anomaly detection with agent-based reasoning provides a powerful and scalable solution for intelligent system monitoring and automated operations.

8.2 FUTURE SCOPE

Despite being highly efficient, there are multiple paths to improving the proposed solution and applying it even more effectively.

One of the potential ways to enhance the solution is to introduce real-time streaming processing capabilities into it. Now the system works only with pre-processed data, but applying real-time stream processing would allow detecting anomalies in near real-time and respond to them immediately, which is especially useful in live environments.

The next possible way of enhancement is introducing adaptive or even online learning techniques into the algorithm. With such an approach, the model would be capable of training itself according to incoming data continuously, which would allow for better detection of changing patterns in the system's work.

As regards the agentic pipeline, there are opportunities for its extension via employing more advanced reasoning approaches, like reinforcement learning or optimization-based methods that would improve the accuracy of the model's recommendations.

Moreover, one can incorporate more kinds of data in the pipeline: not only the application logs but also such data as system metrics, network logs, performance counters, etc. Combining all such kinds of data in one pipeline would improve the efficiency of detecting anomalies even more.

One way to increase scalability is through employing the system as a distributed system in which various pipeline components run on multiple nodes. This approach would allow dealing with massive enterprise systems that produce logs in bulk volumes.

Another promising trend is introducing explainable AI mechanisms to further enhance transparency. Although our system already presents well-structured explanations, using formalized methods of AI explainability might further increase interpretability and credibility of the reasoning process.

Lastly, the suggested approach can be supplemented by implementing automated execution of suggested fixes, i.e., directly applying recommendations without manual intervention. This will allow transitioning towards fully autonomous AIOps systems

that are able to self-heal and optimize themselves automatically.

To summarize, the discussed framework is a good basis for building advanced anomaly detection and analysis systems that have many possibilities for future improvement in terms of processing speed, scalability, and automation.

BIBLIOGRAPHY

- Ali, S., Khan, Z., and Ahmed, F. (2025). A comprehensive study of machine learning techniques for log anomaly detection. *Empirical Software Engineering*, 30(2):10669.
- Chen, S., Zhang, H., and Lyu, M. R. (2022). Bert-log: Anomaly detection for system logs based on bert. *Computer Science*, 36(4):1–10.
- De la Cruz Cabello, M., Sales, T. P., and Machado, M. R. (2025). Aiops for log anomaly detection in the era of llms: A systematic literature review. *Intelligent Systems with Applications*, 28:200608.
- Du, M., Li, F., Zheng, G., and Srikumar, V. (2017). Deeplog: Anomaly detection and diagnosis from system logs through deep learning. In *Proceedings of the 2017 ACM SIGSAC Conference on Computer and Communications Security*, pages 1285–1298. ACM.
- Guo, H., Yuan, S., and Wu, X. (2021). Logbert: Log anomaly detection via bert. *arXiv preprint arXiv:2103.04475*.
- Huang, S., Zhang, H., and Lyu, M. R. (2020). Hierarchical transformers for anomaly detection in system logs. *IEEE Transactions on Network and Service Management*, 17(4):2064–2078.
- Landauer, M., Onder, S., Skopik, F., and Wurzenberger, M. (2023). Deep learning for anomaly detection in log data: A survey. *Machine Learning with Applications*, 13:100470.
- Le, V.-H., Zhang, H., and Lyu, M. R. (2022). Log-based anomaly detection with deep learning: A survey. *arXiv preprint arXiv:2202.04301*.
- Li, W., Zhang, H., and Wang, J. (2025). System log anomaly detection based on contrastive learning and retrieval-augmented methods. *Scientific Reports*, 15(1):22208.
- Meng, W., Liu, Y., Zhu, Y., Zhang, S., Zhang, X., and Lyu, M. R. (2019). Loganomaly: Unsupervised detection of sequential and quantitative anomalies in unstructured logs.

- In *Proceedings of the 28th International Joint Conference on Artificial Intelligence*, pages 4739–4745. IJCAI.
- Techaviseschai, A., Tarnpradab, S., Barroso, V. C., and Phunchongharn, P. (2025). A real-time semi-supervised log anomaly detection framework for alic e o² facilities. *Applied Sciences*, 15(11):5901.
- Tian, G., Li, X., Wang, Y., and Liu, W. (2023). Cldtlog: System log anomaly detection method based on contrastive learning and dual-objective tasks. *Sensors*, 23(11):5042.
- Wu, N., Zhang, L., and Chen, R. (2025). Dual encoder contrastive learning with augmented views for log anomaly detection. *arXiv preprint arXiv:2511.0387*.
- Yang, L., Chen, Z., Zhang, H., and Lyu, M. R. (2021). Plelog: Semi-supervised log-based anomaly detection via probabilistic label estimation. In *Proceedings of the 43rd International Conference on Software Engineering*, pages 230–241. IEEE.
- Yin, Z., Li, M., and Chen, X. (2024). Semi-supervised log anomaly detection based on hierarchical clustering and bidirectional temporal convolution network. *Computers & Security*, 132:103109.
- Zhang, C., Liu, Y., and Wang, H. (2025a). Layerlog: Log sequence anomaly detection based on layered attention mechanism. In *Proceedings of the 2025 IEEE International Conference on Big Data*, pages 1234–1243. IEEE.
- Zhang, L., Wang, M., and Chen, H. (2025b). A survey of aiops in the era of large language models. *arXiv preprint arXiv:2507.12472*.
- Zhang, L., Wang, Q., and Liu, J. (2025c). E-log: Fine-grained elastic log-based anomaly detection using adaptive thresholding. *IEEE Transactions on Services Computing*, 18(5):789–800.
- Zhang, X., Xu, D., Zhang, H., and Lyu, M. R. (2019). Robust log-based anomaly detection on unstable log data. In *Proceedings of the 2019 27th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering*, pages 807–817. ACM.

Zhao, X., Wang, Y., Liu, M., and Zhang, W. (2025). Fusionlog: Cross-system log-based anomaly detection via semantic routing and knowledge distillation. *arXiv preprint arXiv:2511.05878*.

Appendix A- Sample Code

DATA LOADING

```
import pandas as pd
import numpy as np
import pickle

# Load structured datasets
block_lookup = pd.read_csv("data/block_lookup.csv", index_col=0)
seq_lookup = pd.read_csv("data/seq_lookup.csv", index_col=0)

# Load template dictionary
with open("data/template_dict.pkl", "rb") as f:
    template_dict = pickle.load(f)
```

FEATURE EXTRACTION

```
def get_block_features(block_id):
    X = block_lookup.loc[block_id].values.astype(float)
    S = seq_lookup.loc[block_id].values.astype(float)
    return X, S

def normalize_sequence(S, max_event=29):
    S = S / max_event
    return S

def preprocess_features(block_id):
    X, S = get_block_features(block_id)

    # Normalize sequence
    S = normalize_sequence(S)
```

```
return X, S
```

MODEL LOADING

```
from tensorflow.keras.models import load_model
```

```
# Load LOF model
```

```
with open("models/lof_model.pkl", "rb") as f:
```

```
    lof_model = pickle.load(f)
```

```
# Load scaler
```

```
with open("models/scaler.pkl", "rb") as f:
```

```
    scaler = pickle.load(f)
```

```
# Load TCN model
```

```
tcn_model = load_model("models/tcn_model.h5")
```

```
# Load hybrid config
```

```
with open("models/hybrid_config.pkl", "rb") as f:
```

```
    config = pickle.load(f)
```

```
THRESHOLD = config["threshold"]
```

```
W_LOF = config["w_lof"]
```

```
W_TCN = config["w_tcn"]
```

DETECTION LAYER

```
def compute_lof_score(X):
```

```
    X_scaled = scaler.transform([X])
```

```
    score = -lof_model.decision_function(X_scaled)[0]
```

```
    score = np.clip(score, 0, 1)
```

```
    return score
```

```
def compute_tcn_score(S):
    S = np.array(S).reshape(1, 50, 1)
    score = tcn_model.predict(S, verbose=0)[0][0]
    return score
```

HYBRID SCORING AND CLASSIFICATION

```
def compute_hybrid_score(lof_score, tcn_score):
    return (W_LOF * lof_score) + (W_TCN * tcn_score)

def classify_block(hybrid_score):
    if hybrid_score > THRESHOLD:
        return "Anomaly"
    return "Normal"
```

DETECTION PIPELINE

```
def detect_block(block_id):
    X, S = preprocess_features(block_id)

    lof_score = compute_lof_score(X)
    tcn_score = compute_tcn_score(S)

    hybrid_score = compute_hybrid_score(lof_score, tcn_score)
    label = classify_block(hybrid_score)

    return {
        "block_id": block_id,
        "lof_score": lof_score,
        "tcn_score": tcn_score,
        "hybrid_score": hybrid_score,
        "label": label
    }
```

AGENTIC AI PIPELINE IMPLEMENTATION

```
import numpy as np
import pandas as pd
import pickle
import time
from tensorflow.keras.models import load_model
import faiss

# =====
# LOAD DATA AND MODELS
# =====

block_lookup = pd.read_csv("data/block_lookup.csv", index_col=0)
seq_lookup = pd.read_csv("data/seq_lookup.csv", index_col=0)

with open("data/template_dict.pkl", "rb") as f:
    template_dict = pickle.load(f)

with open("models/lof_model.pkl", "rb") as f:
    lof_model = pickle.load(f)

with open("models/scaler.pkl", "rb") as f:
    scaler = pickle.load(f)

tcn_model = load_model("models/tcn_model.h5")

with open("models/hybrid_config.pkl", "rb") as f:
    config = pickle.load(f)

THRESHOLD = config["threshold"]
W_LOF = config["w_lof"]
```

```

W_TCN = config["w_tcn"]

# =====
# FEATURE RETRIEVAL
# =====

def get_block_features(block_id):
    X = block_lookup.loc[block_id].values
    S = seq_lookup.loc[block_id].values
    return X, S

# =====
# DETECTION LAYER
# =====

def compute_lof_score(X):
    X_scaled = scaler.transform([X])
    score = -lof_model.decision_function(X_scaled)[0]
    return np.clip(score, 0, 1)

def compute_tcn_score(S):
    S = np.array(S).reshape(1, 50, 1)
    return tcn_model.predict(S, verbose=0)[0][0]

def compute_hybrid_score(lof_score, tcn_score):
    return (W_LOF * lof_score) + (W_TCN * tcn_score)

def detect_block(block_id):
    X, S = get_block_features(block_id)

    lof_score = compute_lof_score(X)
    tcn_score = compute_tcn_score(S)

```

```

hybrid_score = compute_hybrid_score(lof_score, tcn_score)

label = "Anomaly" if hybrid_score > THRESHOLD else "Normal"

return {
    "block_id": block_id,
    "lof_score": lof_score,
    "tcn_score": tcn_score,
    "hybrid_score": hybrid_score,
    "label": label
}

# =====
# STATE INITIALIZATION
# =====

def initialize_state(block_id):
    return {
        "block_id": block_id,
        "scores": {},
        "severity": None,
        "log_analysis": None,
        "root_cause": None,
        "retrieved_cases": None,
        "remediation": None,
        "latency": {}
    }

# =====
# AGENT 1: MONITORING
# =====

```

```

def agent_monitoring(state):
    start = time.time()

    result = detect_block(state["block_id"])
    state["scores"] = result

    state["latency"]["monitoring"] = time.time() - start
    return state

# =====
# AGENT 2: SEVERITY
# =====

def agent_severity(state):
    start = time.time()

    score = state["scores"]["hybrid_score"]

    if score > 0.9:
        severity = "CRITICAL"
    elif score > 0.75:
        severity = "HIGH"
    elif score > 0.6:
        severity = "MEDIUM"
    else:
        severity = "LOW"

    state["severity"] = severity

    state["latency"]["severity"] = time.time() - start
    return state

```

```

# =====
# AGENT 3: LOG ANALYSIS
# =====

def agent_log_analysis(state):
    start = time.time()

    block_id = state["block_id"]
    seq = seq_lookup.loc[block_id].values

    decoded_events = []
    for e in seq:
        if e == 0:
            continue
        event_key = f"E{int(e)}"
        decoded_events.append(template_dict.get(event_key, "UNKNOWN"))

    state["log_analysis"] = decoded_events[:20]

    state["latency"]["log_analysis"] = time.time() - start
    return state

# =====
# AGENT 4: MEMORY RETRIEVAL (FAISS)
# =====

def agent_memory(state, index, embedder):
    start = time.time()

    query_text = " ".join(state["log_analysis"])
    query_vec = embedder.encode([query_text])

```

```

D, I = index.search(np.array(query_vec).astype("float32"), k=3)

state["retrieved_cases"] = I.tolist()

state["latency"]["memory"] = time.time() - start
return state

# =====
# AGENT 5: ROOT CAUSE
# =====

def agent_root_cause(state):
    start = time.time()

    severity = state["severity"]
    logs = state["log_analysis"]

    if "error" in " ".join(logs).lower():
        cause = "Disk failure or replication issue"
    elif severity == "CRITICAL":
        cause = "Resource contention or node failure"
    else:
        cause = "Transient anomaly"

    state["root_cause"] = cause

    state["latency"]["root_cause"] = time.time() - start
    return state

# =====
# AGENT 6: REMEDIATION

```

```

# =====

def agent_remediation(state):
    start = time.time()

    severity = state["severity"]

    if severity == "CRITICAL":
        action = [
            "Restart affected nodes",
            "Rebalance HDFS cluster",
            "Check disk health"
        ]
    elif severity == "HIGH":
        action = [
            "Check logs",
            "Monitor disk usage"
        ]
    else:
        action = [
            "Observe system",
            "No immediate action required"
        ]

    state["remediation"] = action

    state["latency"]["remediation"] = time.time() - start
    return state

# =====
# FULL PIPELINE EXECUTION
# =====

```

```

def run_agentic_pipeline(block_id, index, embedder):
    state = initialize_state(block_id)

    # Step 1: Detection
    state = agent_monitoring(state)

    if state["scores"]["label"] == "Normal":
        return state

    # Step 2: Reasoning Pipeline
    state = agent_severity(state)
    state = agent_log_analysis(state)
    state = agent_memory(state, index, embedder)
    state = agent_root_cause(state)
    state = agent_remediation(state)

    return state

# =====
# EXECUTION EXAMPLE
# =====

if __name__ == "__main__":
    block_id = "blk_12345"

    # Dummy FAISS index + embedder assumed loaded
    result = run_agentic_pipeline(block_id, index, embedder)

    print(result)

```